FIAP

MGPEB

MÓDULO DE GERENCIAMENTO DE POUSO E ESTABILIZAÇÃO DE BASE

DESAFIO FASE 2

INTEGRANTES

Emanuel Domingues

João Silva

SEC 01 Introdução e objetivo da atividade

Este relatório apresenta o protótipo do MGPEB - Módulo de Gerenciamento de Pouso e Estabilização de Base - criado para a missão fictícia Aurora Siger. A proposta foi organizar a chegada de módulos em Marte usando conceitos estudados na fase: portas lógicas, funções booleanas, estruturas lineares, algoritmos de busca e ordenação, modelagem matemática, evolução da computação e princípios ESG.

A ideia central do projeto foi simular um sistema simples, mas coerente. Em vez de apenas listar comandos em Python, o relatório explica por que cada decisão foi tomada: o que entra na fila, quando um pouso é autorizado, quando um módulo precisa de alerta e como os dados podem ser consultados depois da simulação.

SEC 02 Cenário de pouso e módulos definidos

A base Aurora Siger está em fase inicial de instalação. Por isso, cada módulo que chega à órbita precisa ser avaliado antes de pousar. O sistema recebe módulos de habitação, energia, laboratório, logística e suporte médico. Para cada um deles foram definidos atributos básicos: prioridade, combustível, massa, criticidade e horário estimado de chegada.

Módulo	Tipo	Prior.	Comb.	Massa	Crit.	Horário	Situação
HAB-01	Habitação	1	72%	18,5 t	5	08h	Pousou
ENER-01	Energia	2	55%	12,0 t	4	09h	Pousou
LAB-01	Laboratório Científico	3	88%	9,5 t	3	11h	Pousou
LOG-01	Logística	4	30%	22,0 t	2	14h	Alerta
MED-01	Suporte Médico	2	91%	7,8 t	5	10h	Pousou
ENER-02	Energia	2	45%	11,5 t	4	13h	Pousou
LOG-02	Logística	5	20%	25,0 t	1	16h	Alerta

O critério de segurança adotado considera combustível mínimo de 40% e massa máxima de 24 toneladas. Assim, LOG-01 e LOG-02 entram em alerta por combustível baixo. O LOG-02 também ultrapassa a massa máxima, mas no código a primeira triagem de falha registrada é o combustível, enviando o módulo para a pilha de alertas.

SEC 03 Estruturas de dados utilizadas

Para representar a operação, foram usadas estruturas simples e fáceis de explicar. Essa escolha é importante porque o objetivo da atividade não era criar um sistema industrial completo, mas mostrar domínio dos conceitos fundamentais.

Estrutura	Tipo em Python	Como funciona	Uso no projeto
Fila de pouso	collections.deque	FIFO: o primeiro que entra é o primeiro a ser processado.	Guarda os módulos aguardando autorização de pouso.
Lista de pousados	list	Armazena itens em sequência e permite percorrer os resultados.	Registra os módulos que pousaram com sucesso.
Pilha de alertas	list	LIFO: o último alerta entra no topo e é atendido primeiro.	Recebe módulos com combustível abaixo do mínimo.
Lista de espera	list	Lista auxiliar para casos bloqueados por ambiente ou sensores.	Ficou vazia na simulação, pois atmosfera, área e sensores estavam OK.

No código, a fila principal é carregada com todos os módulos. Em seguida, cada módulo é retirado com popleft(), analisado pelas regras booleanas e encaminhado para uma das listas auxiliares. Esse fluxo deixa a simulação fácil de acompanhar e de testar.

SEC 04 Regras de decisão e portas lógicas

A autorização de pouso foi modelada como uma decisão booleana. O módulo só pode pousar quando todos os critérios principais são verdadeiros: combustível suficiente, massa permitida, atmosfera favorável, área disponível e sensores operacionais.

Expressão booleana adotada:
AUTORIZADO = (combustível_ok AND massa_ok) AND (atmosfera_ok AND área_ok) AND sensores_ok

Variável	Condição testada	Origem da informação
combustível_ok	Combustível maior ou igual a 40%.	Atributo do módulo.
massa_ok	Massa menor ou igual a 24 toneladas.	Atributo do módulo.
atmosfera_ok	Condições atmosféricas favoráveis.	Condição geral da simulação.
área_ok	Área de pouso disponível.	Condição geral da simulação.
sensores_ok	Sensores operacionais.	Condição geral da simulação.

Essa regra representa uma porta AND composta. Na prática, basta uma condição falsa para bloquear o pouso. Esse comportamento é adequado para um sistema crítico, porque ele prioriza segurança em vez de tentar pousar com dados incompletos ou fora do limite.

SEC 05 Trecho do protótipo Python

O trecho abaixo concentra a lógica principal do MGPEB. Ele mostra como as verificações do módulo e as condições externas são combinadas para chegar à decisão final.

```
def autorizar_pouso(modulo, atm_ok, area_ok, sensores_ok):
    combustivel_ok = verificar_combustivel_minimo(modulo)
    massa_ok = verificar_massa_maxima(modulo)

    seguranca_modulo = combustivel_ok and massa_ok
    condicoes_externas = atm_ok and area_ok
    decisao_final = seguranca_modulo and condicoes_externas and sensores_ok

    return decisao_final, combustivel_ok, massa_ok
```

Depois da decisão, o laço de simulação envia o módulo para a lista correta: lista de pousados, pilha de alertas ou lista de espera. O uso de IF, ELIF e ELSE deixa claro o caminho tomado em cada caso.

SEC 06 Busca e ordenação

Após a simulação, os dados são reaproveitados para demonstrar algoritmos clássicos. Essa parte não serve apenas para enfeitar o código: ela mostra como a equipe conseguiria consultar rapidamente informações importantes da operação.

Algoritmo	Uso no projeto	Resultado observado
Busca linear por tipo	Localizar módulos de uma mesma categoria.	Energia: ENER-01 e ENER-02.
Busca do menor combustível	Encontrar o módulo em pior situação de abastecimento.	LOG-02 com 20%.
Busca da maior prioridade	Identificar a carga mais urgente.	HAB-01, prioridade 1.
Busca binária por horário	Encontrar horário em lista já ordenada.	MED-01 encontrado às 10h.
Bubble Sort	Ordenar por prioridade.	Prioridade 1 aparece antes das demais.
Selection Sort	Ordenar combustível em ordem crescente.	LOG-02 e LOG-01 aparecem no início.
Insertion Sort	Ordenar por horário.	Sequência de 08h até 16h.

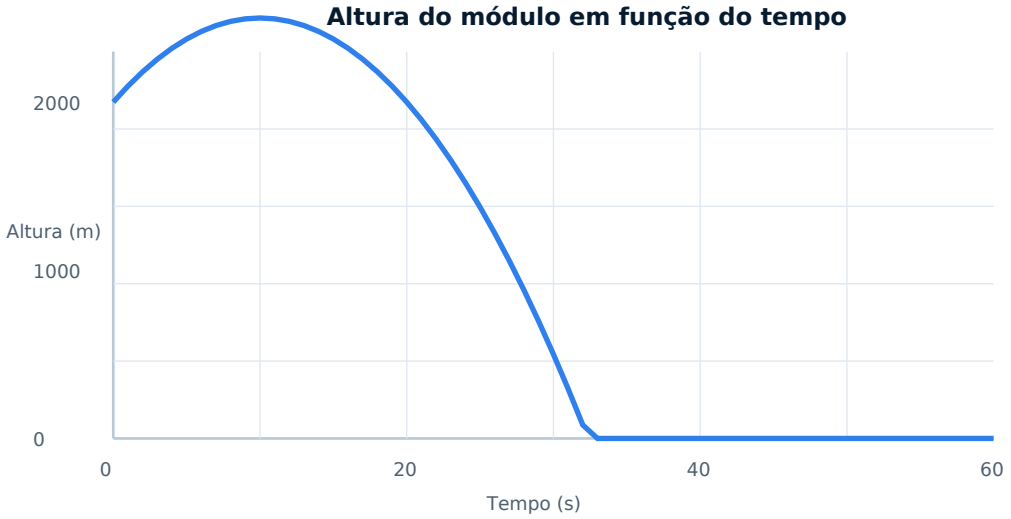
A escolha por algoritmos simples, como Bubble Sort e Selection Sort, faz sentido porque o conjunto de dados é pequeno. Em um sistema real com milhares de registros, algoritmos mais eficientes seriam recomendados. Para a proposta da atividade, a prioridade foi demonstrar claramente o funcionamento.

SEC 07 Modelagem matemática aplicada ao pouso

Para ligar o código a uma situação física, foi adotada uma função quadrática de altura ao longo do tempo de descida. A ideia é mostrar como a equipe poderia usar matemática para decidir quando iniciar sistemas de frenagem.

Função adotada:
$h(t) = -5t^2 + 100t + 2000$

Onde: $h(t)$ é a altura do módulo em metros; t é o tempo em segundos; $h(0) = 2000$ m representa a altura inicial aproximada. Como o termo de t^2 é negativo, a curva indica perda de altura com aceleração ao longo da descida.



O gráfico mostra que a altura cai rapidamente. Por isso, o sistema precisa decidir o momento certo para acionar retrofoguetes ou paraquedas. Se esperar demais, o risco de impacto aumenta; se acionar cedo demais, desperdiça combustível. Essa relação ajuda a justificar por que sensores e limites de segurança aparecem como condições obrigatórias na lógica do MGPEB.

SEC 08 Computação embarcada e contexto histórico

O MGPEB também pode ser entendido como uma versão didática de sistemas embarcados usados em missões espaciais. Desde os primeiros computadores de grande porte até os processadores resistentes à radiação, a evolução da computação tornou possível colocar sistemas confiáveis em ambientes extremos.

Primeiros computadores: equipamentos como ENIAC e outros computadores iniciais mostraram que cálculos automatizados podiam apoiar decisões complexas, mesmo ocupando muito espaço e consumindo muita energia.

Circuitos integrados e microprocessadores: a redução de tamanho e consumo abriu caminho para computadores embarcados, que executam tarefas específicas dentro de veículos, sondas e rovers.

Missões espaciais: em Marte, memória, energia, processamento e tolerância à radiação são limitados. Por isso, o projeto valoriza regras claras, estruturas simples e operações previsíveis.

Nesse sentido, o uso de listas, pilhas, filas e funções booleanas não é apenas um requisito acadêmico. Ele representa uma decisão de projeto: preferir soluções simples, auditáveis e adequadas a um ambiente onde cada erro pode comprometer a missão.

SEC 09 ESG e governança da base Aurora Siger

Mesmo sendo uma simulação, a base Aurora Siger precisa considerar responsabilidade ambiental, social e de governança. Em Marte, qualquer decisão sobre pouso, energia ou uso de recursos pode afetar a segurança da tripulação e a preservação científica do ambiente.

Dimensão	Aplicação na base Aurora Siger
Ambiental	Escolher áreas de pouso com menor impacto geológico, evitar contaminação de zonas sensíveis e reaproveitar recursos locais sempre que possível.
Social	Priorizar módulos ligados à sobrevivência, como habitação, energia e suporte médico, garantindo segurança para todos os tripulantes e promovendo bem-estar da equipe.
Governança	Registrar as decisões do MGPEB em logs claros, revisar critérios técnicos periodicamente e permitir auditoria das autorizações e bloqueios de pouso.

Ao integrar esses princípios, o MGPEB deixa de ser apenas um conjunto de regras técnicas e passa a apoiar uma operação ética, transparente e alinhada aos objetivos de exploração e ciência.

SEC 10 Resultado da simulação

A execução do código carregou 7 módulos na fila. Como as condições atmosféricas, a área de pouso e os sensores estavam favoráveis, os módulos com combustível e massa dentro do limite foram autorizados. O resultado final foi: 5 módulos pousados, 2 alertas na pilha e 0 módulos em espera.

Grupo	Módulos
Pousados	HAB-01, ENER-01, LAB-01, MED-01 e ENER-02.
Alertas	LOG-02 e LOG-01, ambos por combustível abaixo de 40%.
Em espera	Nenhum módulo ficou em espera neste ciclo.

SEC 11 Anexo visual da execução do programa

As imagens abaixo mostram a execução real do programa no terminal. Elas ajudam a comprovar que o código foi testado e que os resultados apresentados no relatório saem da simulação.

Execução inicial da simulação

```
PS C:\Users\emanu\OneDrive\Documents\FIAP-FACUL\FASE 2\DESAFIO-FASE2-FIAPON> & C:\Users\emanu\AppData\Local\Pyt
thon.exe "c:/Users/emanu/OneDrive/Documents/FIAP-FACUL/FASE 2/DESAFIO-FASE2-FIAPON/mgpeb.py"
[INIT] 7 módulos carregados na fila de pouso.

=====================================================
   SIMULAÇÃO DE POUSO - MISSÃO AURORA SIGER
=====================================================
Condições atmosféricas : ✓ Favoráveis
Área de pouso          : ✓ Disponível
Sensores               : ✓ Operacionais
=====================================================

[01] Processando: HAB-01 (Habitação)
     Prioridade: 1 | Combustível: 72% | Massa: 18.5t | Horário: 8h
     ✓ POUSO AUTORIZADO — HAB-01 pousou com sucesso.

[02] Processando: ENER-01 (Energia)
     Prioridade: 2 | Combustível: 55% | Massa: 12.0t | Horário: 9h
     ✓ POUSO AUTORIZADO — ENER-01 pousou com sucesso.

[03] Processando: LAB-01 (Laboratório Científico)
     Prioridade: 3 | Combustível: 88% | Massa: 9.5t | Horário: 11h
     ✓ POUSO AUTORIZADO — LAB-01 pousou com sucesso.

[04] Processando: LOG-01 (Logística)
     Prioridade: 4 | Combustível: 30% | Massa: 22.0t | Horário: 14h
     ⚠ ALERTA DE COMBUSTÍVEL — LOG-01 (30% < 40%) empilhado para intervenção urgente.

[05] Processando: MED-01 (Suporte Médico)
     Prioridade: 2 | Combustível: 91% | Massa: 7.8t | Horário: 10h
     ✓ POUSO AUTORIZADO — MED-01 pousou com sucesso.
```

Relatório final de pouso

```
=============================================================
   RELATÓRIO FINAL DE POUSO
=============================================================

✓ MÓDULOS POUSADOS (5):
   - HAB-01 (Habitação), Prioridade 1
   - ENER-01 (Energia), Prioridade 2
   - LAB-01 (Laboratório Científico), Prioridade 3
   - MED-01 (Suporte Médico), Prioridade 2
   - ENER-02 (Energia), Prioridade 2

⚠ ALERTAS NA PILHA (2) [atende o último primeiro]:
   - LOG-02 (Logística), Combustível: 20%
   - LOG-01 (Logística), Combustível: 30%

✗ MÓDULOS EM ESPERA (0):
```

SEC 11 Anexo visual da execução do programa - continuação

Busca e ordenação

```
DEMONSTRAÇÃO DE ALGORITMOS DE BUSCA

🔍 Módulo com MENOR combustível: LOG-02 (20%)
🔍 Módulo com MAIOR prioridade : HAB-01 (Prioridade 1)
🔍 Módulos do tipo 'Energia'   : ['ENER-01', 'ENER-02']

DEMONSTRAÇÃO DE ALGORITMOS DE ORDENAÇÃO

📋 Fila reordenada por PRIORIDADE (Bubble Sort):
   [1] HAB-01 (Habitação)
   [2] ENER-01 (Energia)
   [2] MED-01 (Suporte Médico)
   [2] ENER-02 (Energia)
   [3] LAB-01 (Laboratório Científico)
   [4] LOG-01 (Logística)
   [5] LOG-02 (Logística)

📋 Fila reordenada por COMBUSTÍVEL ↑ (Selection Sort):
   [20%] LOG-02
   [30%] LOG-01
   [45%] ENER-02
   [55%] ENER-01
   [72%] HAB-01
   [88%] LAB-01
   [91%] MED-01
```

Busca binária e encerramento

```
📋 Fila reordenada por HORÁRIO (Insertion Sort):
   [8h] HAB-01
   [9h] ENER-01
   [10h] MED-01
   [11h] LAB-01
   [13h] ENER-02
   [14h] LOG-01
   [16h] LOG-02

🔍 Busca Binária por horário 10h: encontrado 'MED-01' no índice 2.

============================================================
   FIM DA SIMULAÇÃO MGPEB — AURORA SIGER
============================================================
```

SEC 12 Anexo - estruturas de dados e fechamento

O anexo reforça como cada estrutura foi aplicada no código. A fila organiza a chegada dos módulos; a pilha concentra alertas urgentes; as listas guardam o histórico da operação. Essa separação deixa o código mais legível e facilita a análise posterior.

Operação	Exemplo no código	Interpretação
Inserir na fila	fila_pouso.append(m)	Coloca cada módulo no fim da fila de pouso.
Retirar da fila	fila_pouso.popleft()	Processa o primeiro módulo cadastrado.
Registrar pouso	lista_pousados.append(modulo)	Guarda módulo autorizado com sucesso.
Empilhar alerta	pilha_alertas.append(modulo)	Envia módulo crítico para atendimento urgente.
Consultar alertas	reversed(pilha_alertas)	Mostra o último alerta primeiro, respeitando a lógica LIFO.

Concluindo, o protótipo atende ao objetivo da atividade porque conecta a lógica de programação a um problema de engenharia simulado. O sistema é simples, porém completo o suficiente para demonstrar tomada de decisão, organização de dados, busca, ordenação, modelagem matemática e reflexão sobre o uso responsável da tecnologia.